



Cairo University

Faculty of Economics and Political Science

Socio-Computing Department

C201: Computer Applications in Social Sciences II

MODULE 2: PYTHON FUNDAMENTALS

LECTURE 2_1

VARIABLES, OPERATORS, EXPRESSIONS, AND CONDITION (IF...ELSE)

MODULE 2: LEARNING OBJECTIVES



For many experts, the world of advanced analytics is unlocked using Python. This part focuses on fundamental programming concepts and algorithmic thinking.

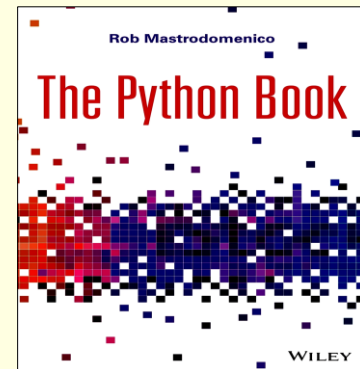
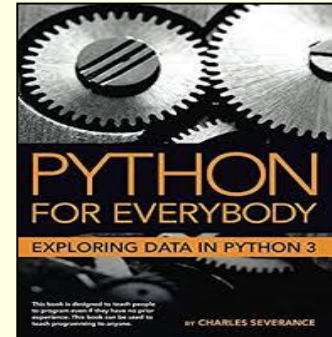
By the end of this module, you will

- Write and execute Python programs using different development environments.
- Learn core Python concepts, such as variables, data types, data structures, conditional statements, and loops.
- Use functions and modular programming to write reusable code.
- Learn debugging techniques to identify and fix programming errors.

Module 2: Python Fundamentals

Outlines

- ❑ Python Programming Language: Why Python? Best Python IDEs, Write your first program
- ❑ Variables, Operators, Expressions, and Conditioning (if...Else) (this lecture: [Lecture 2_1](#))
- ❑ Loops (For and While Statements) ([Lecture 2_2](#))
- ❑ Functions, Modules and Packages ([Lecture 2_3](#))
- ❑ Python Data Structures (Lists, Tuples, Dictionaries, and Sets) ([Lectures 2_4_parts 1 and 2](#))



The Python Programming Language

Computer programming is the process of giving instructions to a computer to perform an action or set of actions

Python is a general-purpose interpreted programming language, built on top of C programming language.

Python was first released to the world in **1990** by Guido van Rossum, a Dutch programmer. Its name comes from a 1970s British comedy sketch television show called *Monty Python's Flying Circus*.

Python has a simple syntax, which makes it easily readable and extremely user-friendly. It is widely used in many companies and organizations such as *YouTube, Google, Yahoo,* and *NASA*.

Python is well supported and freely available at www.python.org.



Guido van Rossum

Why Python?

Python is a flexible, general-purpose language that can be used for web development, automation, cryptography, and other tasks.

Python also has several *characteristics* that make it popular amongst developers and engineers. These include:

- **It's Easy to Read**

Python code uses English keywords rather than punctuation. In practice, this means you can identify what the code is designed to do simply by looking at it.

- **It's Open Source**

You can download the source code, modify it, and use it however you want.

- **It's Portable**

Python is a cross-platform language, which means you can run the same code on any operating system with a Python interpreter.

Why Python?

- **It has a Broad Standard Libraries**

The *Python Standard Library* is a collection of *built-in modules*, each providing specific functionality beyond what is included in the core part of Python. For example, the math module provides additional mathematical functions.



These libraries are available for anyone to access. This means that users don't have to write code for every single function—they can access built-in modules that help with issues in everyday programming and more. These libraries are extremely useful in ML and AI. It makes the analysis task efficient and has a great base of algorithms and functions which makes it an excellent language.

Python is the king of the **Data Science Kingdom** because of its excellent libraries, such as *Numpy, Pandas and scikit-learn*.

- **It has Many Career Opportunities**

Python is now applicable in almost every field. It has a satisfying career for everyone. There is a huge demand for Python developers. Python is the most requested skill in data science job demands.

Types of Python Files

There are two general types of Python files: *Python scripts* and *Python notebooks*.

Both types of files can run the same code; however, there are certain situations where one is preferable to the other.

Python Scripts (.py)

- Best for larger projects and production use.
- Easier to debug, reproduce results, and integrate with other software.
- Runs the code as a whole, not in separate parts.

Python Notebooks (.ipynb)

- Ideal for exploration, visualization, and presentations.
- Allows running code in sections with immediate output.
- Supports non-code elements, such as markdown, images, and links, for added context.

Best Python IDEs

Integrated Development Environments (IDEs) are coding tools that make writing, debugging, and testing your code easier. Many provide helpful features like code completion, syntax highlighting, debugging tools, variable explorers, visualization tools, and many other features.

The following list includes a list of the best Python IDEs for Data Science & Machine Learning.

1. **Scientific Python Development Environment (Spyder):** The tool features some advanced editing, debugging, and profiling tools that make coding in Python a lot easier and more efficient. Only supports Python programming
2. **Visual Studio:** an IDE developed by Microsoft, VS offers tips when certain code is written, which makes this a great tool for beginners to learn on.
3. **JupyterLab and Jupyter Notebook:** both a web-based python IDE that supports over 40 programming languages, including Python, R, and other data languages like Julia and Scala.

Best Python IDEs

5. **PyCharm:** preferred by many DS & ML Professional
6. **Google Collaboratory:** a Google's cloud-based notebook environment. Gives you free access to Google hardware, so you don't have to rely on your own machine's hardware for computationally demanding projects. It supports Python, R, and SQL.
7. **DataCamp Workspace:** a cloud-based notebook that allows anyone to analyze data, collaborate, and share insights with their team.
8. **Visual Code:** supports many languages besides Python like C, C#, JavaScript, HTML, CSS, etc. It is also a good platform for beginners.
9. **Others:** Such as DataSpell, Atom, ... etc.

Python Reference Library

<https://docs.python.org/3/library/>

A First Program

Let us start to dive into the amazing world of programming

Step 1: Start IDLE such as Jupyter Notebook

You will immediately see a screen with an empty cell. This is called the *Python shell* (interactive interpreter). It is an interactive window where you can enter lines or blocks of code and run them immediately to see their effect and output.

Step 2: Write a program to display a Message

Python is a very simple language and has a very straightforward syntax.

When we are just starting out with Python, one of the first programs we'll learn is the classic "Hello, World!" program.

It's a simple program that displays the message "Hello, World!" on the screen.

```
print("Hello, World!")          # Out: Hello, World!
```

A First Program

```
print("Hello, World!")      # Out: Hello, World!
```

print() is a built-in function in Python that tells the program to display something on the screen. We need to add the string in parenthesis of print() function that we are displaying on the screen.

"Hello, World!" is a string text that you want to display. Strings are always enclosed in quotation marks.

Step 3: See the Output

You will see the statement `Hello, World!` printed on the screen.

For convenient, the output of any print statement in these presentation will be shown as comment.

A comment in Python is a statement written after the **#** symbol.

Variables

Information in code is stored in variables.

A variable is a *named container* which stores *values* in a *reserved location* in the computer's memory.

If you have the expression

$$X = 8$$

X is the variable **name**, and its stored **value** is 8. The value 8 exists in a specific location in the computer's memory and X **points to that location**.

Variables act as *placeholders for data*. They allow us to store and reuse values in our program. Variables in Python are assigned values using the = operator.

```
x = 10; y = 1.387668; z = True; X = 'Rob'
print(X, x, y, z)
# Out: Rob 10 1.387668 True
```

Note: Python is a case sensitive programming language. Therefore, the variable named capital X is different from the variable named small x .

When assigning a new variable, you should decide about

- *What's the variable's **name**?*
- *What's the variable's **value type**?*

Variables

Naming Conventions

- The variable names can take many forms, although they can only contain **numbers**, **letters** (both upper and lower), and **underscores** (_).
- They must *begin with a letter* or *an underscore*, and they are *case sensitive*.
- Names **can't contain spaces** and **can't start with a number** but may contain numbers after the first character.

*For example, myVar, MyVar, _myVar, MyVar123 are **valid** variable names, but m*var, my-var, my var, 1myVar are **invalid** variable names.*

- Create variable names that remain **meaningful** and easy to reference later to remind yourself and others about what you are storing in that variable.
- *Descriptive names* are **preferable** than *abbreviations* because they make it easier for you and other programmers to read and understand your code.

*For example, **student_name** (underscores used to separate words) or **StudentName** (capitalization of every variable name) are preferable to **sn**. While they could seem excessive, but they will make lot more sense when you go back to your code.*

Variables

Naming Conventions

Names ***can't contain reserved words***

- Reserved words (**Keywords**) are some predefined and reserved words in Python that have special meanings. They are used to define the syntax of the coding.
- All the keywords in Python are written in lowercase except *True* and *False*.
- There are 35 keywords in Python 3.11.

If you execute the below code

```
import keyword
print(keyword.kwlist)
```

You will get the below list of keywords:

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',  
'break', 'class', 'continue', 'def', 'del', 'elif', 'else',  
'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in',  
'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return',  
'try', 'while', 'with', 'yield']
```

Variables

Values Data Types

Variables can store values of any data type.

A data type is an attribute that describes a piece of data based on its values, and the operations it can perform.

In Python, this includes *built-in data types* such as integers and strings and *collection data types (Python Data Structures)* such as *list* and *dictionary*.

Built-in Data Types: *int, float, bool, str*

```
x = 10
y = 1.387668
z = True
name = 'Rob'
type(x), type(y), type(z), type(name) #Out: (int, float, bool, str)
```

Unlike many other programming languages, Python variables do not require explicit declaration of type. The type of the variable is *inferred* based on the value assigned.

Note: Python Data Structures are shortly explained in details after introducing the *operators* next slides.

Variables

Updating Variables

- ❑ One of the most common forms of reassignment is an update where the new value of the variable depends on the old. For example,

$$x = x + 1$$

This means get the current value of x , add one, and then update x with the new value. The new value of x is the old value of x plus 1.

- ❑ If you try to update a variable that doesn't exist, you get an *error* because Python evaluates the expression on the right side of the assignment operator before it assigns the resulting value to the name on the left.
- ❑ Before you can update a variable, you have to initialize it, usually with a simple assignment. In the below example, x was initialized to 6.

```
x = 6           # initialize x
print(x)
x = x + 1       # update x
print(x)
```

Updating a variable by adding 1 is called an *increment*; subtracting 1 is called a *decrement*.

Variables

Updating Variables

- Incrementing and decrementing are such common operations that programming languages often include special syntax for it.
- In Python **+=** is used for incrementing, and **-=** for decrementing.

```
x = 6           # initialize x
print(x)
x += 3          # increment x by 3; same as x = x + 3
print(x)        # 9
x -= 2          # decrement x by 2; same as x = x - 2
print(x)        # 7
```

Python Operators

Operators are symbols that perform operations on objects and values. There are three types of operators in any programming language:

Arithmetic Operators, *comparators Operators*, and *logical Operators*

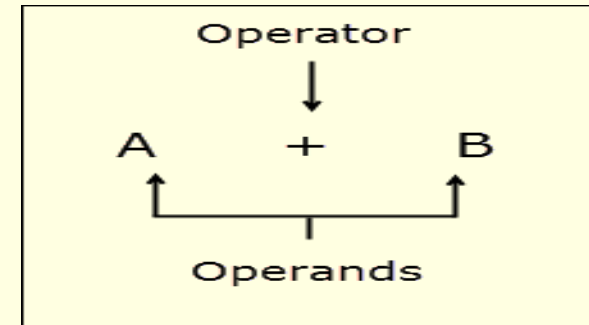
Arithmetic Operators:

Python Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication and division. They applied to numeric types (int and float).

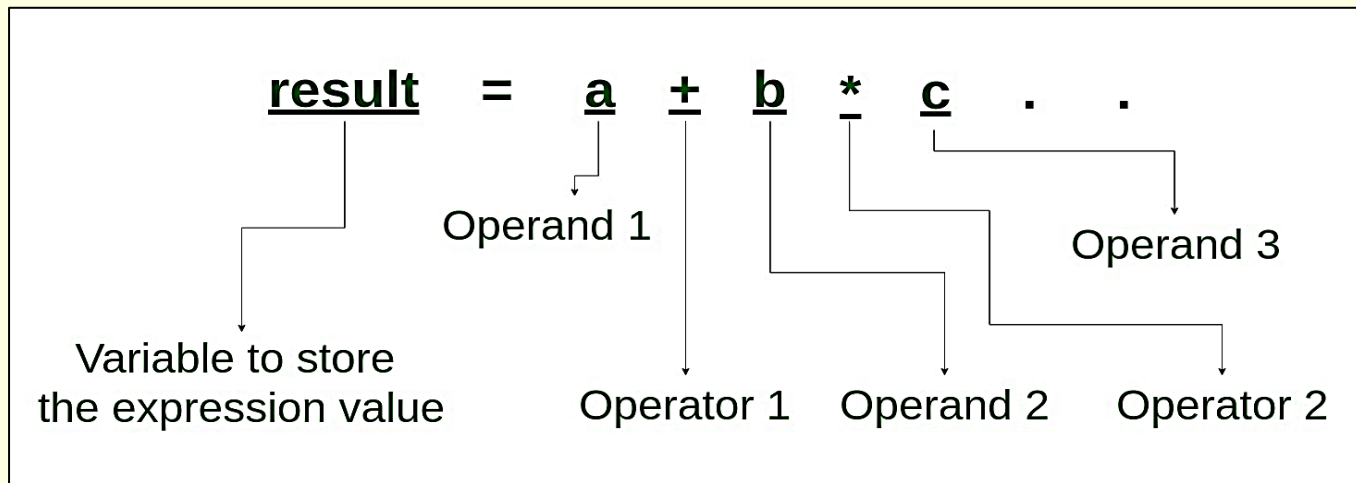
Operator	Meaning	Example	Algebraic
+	Addition	$x + y$	$x + y$
-	Subtraction	$x - y$	$x - y$
*	Multiplication	$x * y$	xy
/	Division (Left divide)	x/y	$\frac{x}{y}$
//	Integer Division	$x//y$	$\lfloor \frac{x}{y} \rfloor$
**	Exponentiation	$x**y$	x^y

Arithmetic Operators

The values the operator is applied to are called *operands*.



An *expression* is a statement which combines *values*, *variables*, and *operators*.



Arithmetic Operators

```
# Example: Apply Different operators on two variables (operands):  
# a and b and calculate two expressions: mean and var  
  
a = 15  
  
b = 4  
  
mean = (a+b)/2  
  
var = (a-mean)**2+ (b-mean)**2  
  
print("Addition:", a + b)           # Out: Addition: 19  
print("Subtraction:", a - b)        # Out: Subtraction: 11  
print("Multiplication:", a * b)     # Out: Multiplication: 60  
print("Division:", a / b)           # Out: Division: 3.75  
print("Floor Division:", a // b)    # Out: Floor Division: 3  
print("Modulus:", a % b)            # Out: Modulus: 3  
print("Exponentiation:", a ** b)    # Out: Exponentiation: 50625  
print("Mean:", mean)                # Out: Mean: 9.5  
print("Variance:", var)             # Out: Variance: 60.5
```

Arithmetic Operators Precedence

In Python, arithmetic operations follow operator precedence, determining the order in which operations are performed. The priority of arithmetic operations (from highest to lowest) is:

Precedence Level	Operator	Description	Associativity	Example	Result
1 (Highest)	()	Parentheses (Override precedence)	Left to Right	(2 + 3) * 4	20
2	**	Exponentiation (Power)	Right to Left	2 ** 3 ** 2	512 (2 ** (3 ** 2))
3	+x, -x	Unary plus and minus	Right to Left	-2	-2
4	*, /, //, %	Multiplication, Division, Floor Division, Modulus	Left to Right	10 / 2 * 3	15
5 (Lowest)	+, -	Addition and Subtraction	Left to Right	10 - 5 + 2	7

Example

	Expression	Step-by-Step Evaluation	Final Result
1	$5 + 3 * 2$	$5 + (3 * 2) \rightarrow 5 + 6$	11
2	$10 - 4 / 2 + 3$	$10 - (4 / 2) + 3 \rightarrow 10 - 2 + 3$	11
3	$8 + 2 ** 3 * 2$	$8 + (2 ** 3) * 2 \rightarrow 8 + 8 * 2 \rightarrow 8 + 16$	24
4	$(6 + 2) * 3 ** 2$	$(8) * (3 ** 2) \rightarrow 8 * 9$	72
5	$18 / 3 + 4 * 2$	$(18 / 3) + (4 * 2) \rightarrow 6 + 8$	14
6	$20 // 3 + 5 \% 3$	$(20 // 3) + (5 \% 3) \rightarrow 6 + 2$	8
7	$15 - 2 * 3 ** 2 / 6$	$15 - (2 * (3 ** 2) / 6) \rightarrow 15 - (2 * 9 / 6) \rightarrow 15 - (18 / 6) \rightarrow 15 - 3$	12
8	$4 + 8 / 2 - 5 * 3$	$4 + (8 / 2) - (5 * 3) \rightarrow 4 + 4 - 15$	-7
9	$30 \% 7 + 2 ** 2 * 3$	$(30 \% 7) + (2 ** 2) * 3 \rightarrow 2 + 4 * 3 \rightarrow 2 + 12$	14
10	$(5 + 3) * 2 ** 3 - 6 / 3$	$(8) * (2 ** 3) - (6 / 3) \rightarrow 8 * 8 - 2 \rightarrow 64 - 2$	62

Comparators Operators

A **Boolean** expression is an expression that is either *True* or *False*.

Comparators (Relational) operators compares two operand values and it either returns *True* or *False* (a Boolean value) according to a condition.

Operator	Definition
>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal
==	Equal
!=	Not equal
is	Is same
Is not	Is not same

```
a = 15
b = 4
print(a > b)           # Out: True
print(a < b)           # Out: False
print(a == b)          # Out: False
print(a != b)          # Out: True
print(a >= b)          # Out: True
print(a <= b)          # Out: False
print (a is b)         # Out: False
print(a is not b)      # Out: True
```

Logical Operators

Computer programs are constantly making decisions based on complex statements by combining **several Boolean** statements that have True/False meaning into a single statement using the logical operators AND, OR, and NOT.

Operator	Description	Example
and	Returns True if both statements are true	$x > 5$ and $x < 10$
or	Returns True if one or both statements is true	$x > 5$ or $x < 4$
not	Reverse the result, returns False if the result is true	$\text{not}(x > 5 \text{ and } x < 10)$

For example,

- In stock market domain, **a company stock will move up** if there is a *positive sentiment toward the company stock* **and** *the company achieved profits in the last year*.

Here the both conditions are necessary happened to have the stock price raised.

- A heart monitoring program might **sound an alarm** if *the pulse is too slow* **or** *the blood pressure is too weak*.

Here one condition is enough to sound the alarm.

Logical Operators

Truth table shows the result of combining any two Boolean expressions using the AND, the OR, or the NOT operators.

(A)	(B)	A AND B	A OR B	NOT A	NOT B
FALSE	FALSE	FALSE	FALSE	TRUE	TRUE
FALSE	TRUE	FALSE	TRUE	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	FALSE	TRUE
TRUE	TRUE	TRUE	TRUE	FALSE	FALSE

The precedence of logical operators in Python is as follows: *NOT, AND, OR*

```
a = True
b = False
print(a and b)      # Out: False
print(a or b)       # Out: True
print(not a)        # Out: False
print(a and not b)  # Out: True
```

Logical Operators Precedence

Assume we have three logical variables: *A*, *B*, and *C*

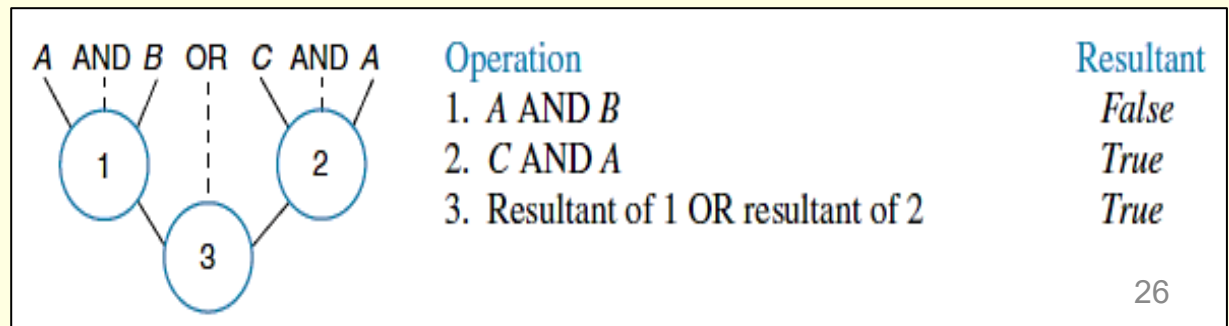
```
A = True; B = False; C = True
print (A and B or C)           #Out: True
print (A and B or not C)       #Out: False
print (A and B or C and A)     #Out: True
print (not A and (B or C))     #Out: False
```

Note

- ❑ Order of execution matters, so always verify how logical conditions are structured correctly.
- ❑ **not** has the highest precedence, followed by **and**. Whereas, **or** has the lowest precedence.
- ❑ Use parentheses to change evaluation order and make expressions clearer.

The expression:

A and B or C and A
is evaluated as follows



Logical Operators Precedence

Assume we have three logical variables: A , B , and C

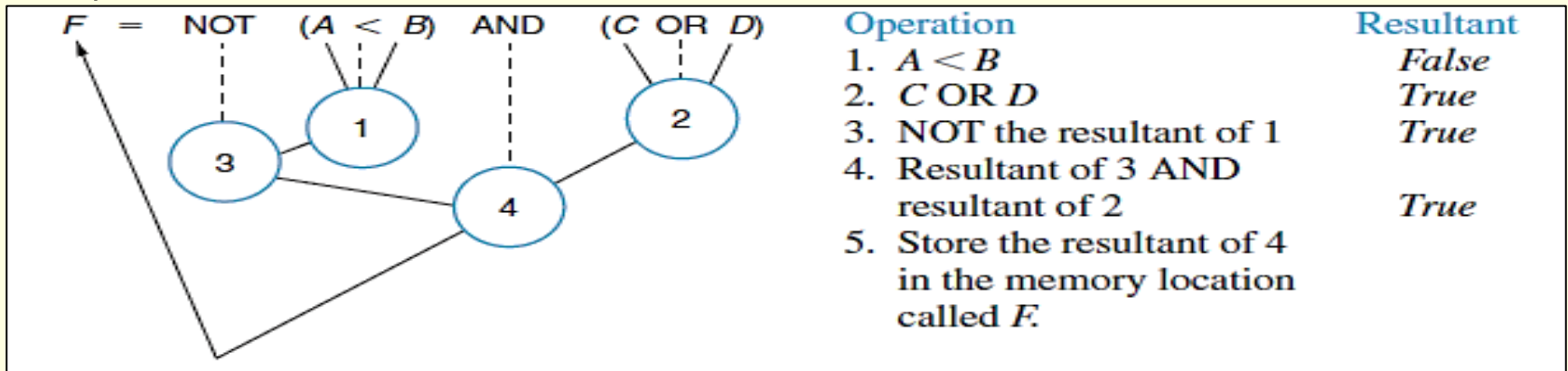
```
A = 4; B = 2; C = True; D = False
```

```
F = not (A < B) and (C or D)
```

```
print (F)
```

#Out: True

The expression: **F** is evaluated as follows



Note:

In Python, the order of evaluation for the three types operators is as follows

Arithmetic $\rightarrow$ Comparison $\rightarrow$ Logical

Meaning arithmetic operations are performed first, comparison operators next, and logical operators last

Example

#	Expression	Step-by-Step Evaluation	Final Result
1	<code>not (8 / 4 == 2) and 3 ** 2 > 8</code>	not (2 == 2) and 9 > 8 → not True and True → False and True	FALSE
2	<code>12 % 5 + 2 * 3 >= 8 and 7 - 2 < 5</code>	(2) + (2 * 3) >= 8 and (7 - 2) < 5 → 2 + 6 >= 8 and 5 < 5 → 8 >= 8 and False → True and False	FALSE
3	<code>not (4 * 2 > 6) or 10 % 3 == 1</code>	not (8 > 6) or (10 % 3) == 1 → not True or 1 == 1 → False or True	TRUE
4	<code>10 - 3 ** 2 < 5 or 8 % 4 == 2</code>	10 - (9) < 5 or (8 % 4) == 2 → 1 < 5 or 0 == 2 → True or False	TRUE
5	<code>not (5 + 2 > 8) and 9 // 3 == 3</code>	not (7 > 8) and (9 // 3) == 3 → not False and 3 == 3 → True and True	TRUE

Example

Write an algorithm to calculate the Final Total Amount (A) of money deposited in a bank using Simple Interest (SI), where SI is given by the formula:

$$\text{Simple interest} = P * N * (R/100)$$

Where P is the principal amount, N is the number of years, and R is the rate of interest.

The Final Total Amount (A) is calculated using the formula: $A = P + SI$

```
P= int(input("Enter The principal Deposit: "))
R = float(input("Rate of interest (0:100) per year: "))
N = int(input("No. of Years: "))
SI = round(P*N*(R/100),2)

A = P + SI

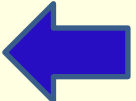
print(f"\nAmount of money after {N} years = {A}")
```

The Output

```
Enter The principal Deposit: 1000
Rate of interest (R) per year: 10
No. of Years: 5
Amount of money after 5 years = 1500.0
```

Outlines

Python Fundamentals

1. Python Programming Language: *Why Python?*
Best Python IDEs, Write your first program
2. Variables, Operators, and Expressions
3. **Condition (if...Else)** 
4. Loops (For and While Statements)
5. Functions, Modules and Packages
6. Python Data Structures (Lists, Tuples, Dictionaries, and Sets)
7. Dealing with Files

Conditional Statement

By default, statements in Python script are executed sequentially from top to bottom. If the processing logic require so, the sequential flow of execution can be altered in two way:

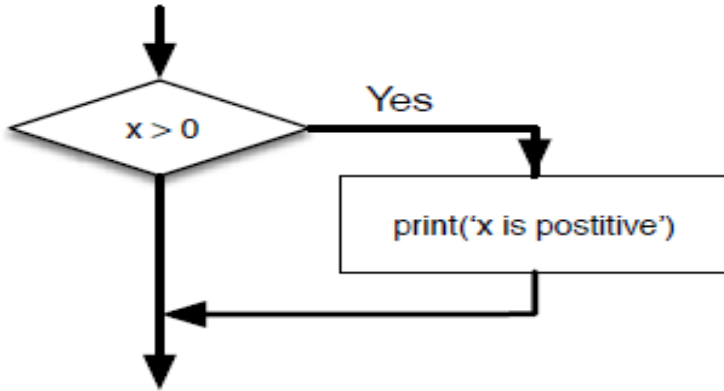
- **Conditional execution:** a block of one or more statements will be executed if a certain expression (condition) is true (*this lecture*).
- **Repetitive execution (Loop):** a block of one or more statements will be repetitively executed as long as a certain expression is true (*next lecture*).

Conditional statements give us the ability to check conditions and change the behavior of the program accordingly.

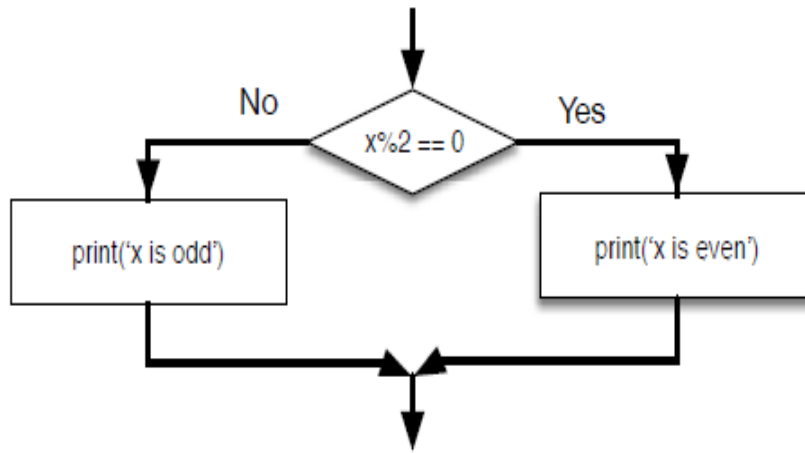
```
# syntax for simple IF condition
if condition:
    this part of code runs for true conditions
else:
    this part of code runs for false conditions
```

Python uses indentation for blocks. Both tabs and spaces are supported, but the standard indentation requires standard Python code to use four spaces.

Example



```
x= input("Enter a numeric number: ")
x= int(x)
if x > 0:
    print(x, ' is a positive number')
```

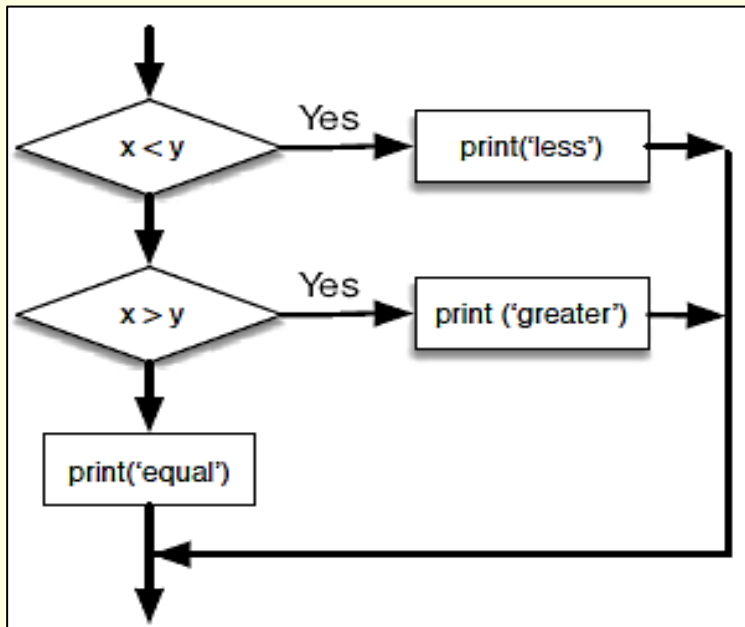


```
x = input("Enter a numeric number: ")
x = int(x)
if (x %2==0):
    print(x, ' is an even number')
else:
    print(x, ' is an odd number')
```

Chained Conditional Statement

Sometimes there are **more than two possibilities**, and we need more than two branches. One way to express a computation like that is a **chained conditional**:

```
# syntax for multiple IF condition
if condition:
    code
elif condition:
    code
else:
    code
```



```
x = input("Enter the first number: ")
x = float(x)
y = input("Enter the first number: ")
y = float(y)

if (x > y):
    print(x, ' is greater than ', y)
elif (y > x):
    print(y, ' is greater than ', x)
else:
    print('The two numbers are equal')
```

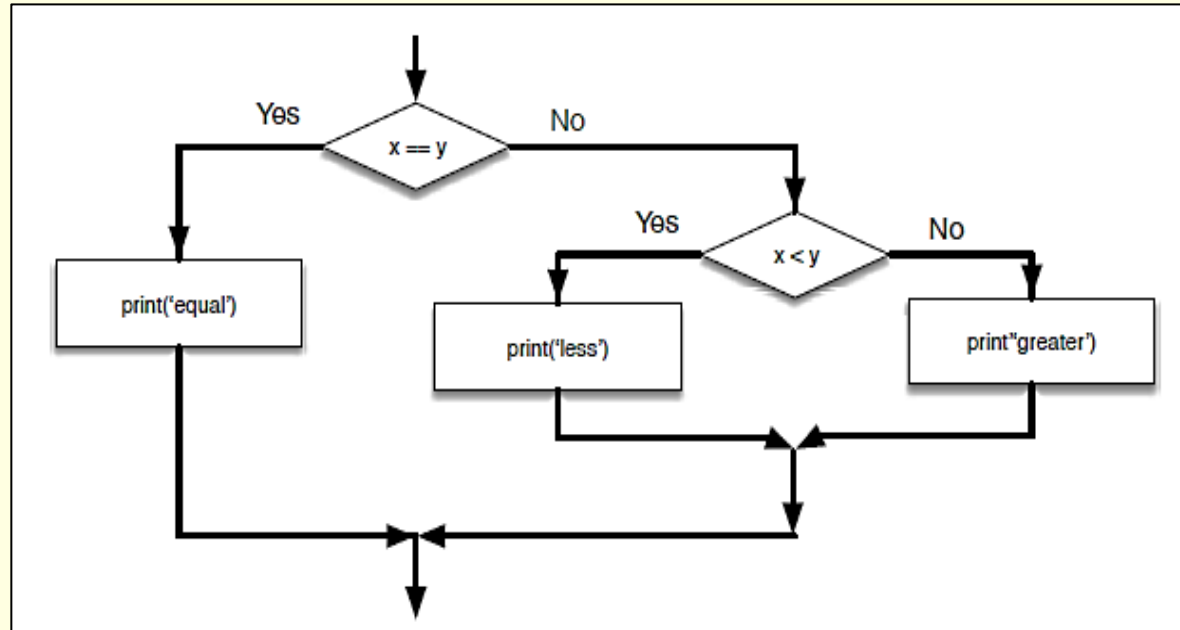
Example

The below example shows *chained compound conditional statement*

```
# Compound If St.
a= input("Enter a numeric number: "); a= float(a)
if a > 0 and a % 2 == 0:
    print('A is an even positive number')
elif a > 0 and a % 2 != 0:
    print('A is an odd positive number')
elif a == 0:
    print('A is zero')
elif a < 0 and a % 2 == 0:
    print('A is an even negative number')
else:
    print('A is an odd negative number ')
```

Nested Conditional Statement

One conditional can also be *nested* within another.



```
x = input("Enter the first number: "); x= float(x)
y = input("Enter the second number: "); y= float(y)

if (x == y):
    print('The two numbers are equal')
else:
    if (x < y):
        print(x, ' is less than ', y)
    else:
        print(y, ' is greater than ', x)
```

Sheet 2 Exercises

1. Check if your birth year is a leap year. A year is a leap year if
 - It is divisible by 400, or
 - It is divisible by 4 but not 100.

Print 'The given year is a Leap Year' or 'The given year is not a Leap Year'

2. A bank offers loans based on age, income, and employment status. The applicant is eligible for a loan based on the following conditions:
 - Must be at least 21 years old
 - Must have an income of at least \$25,000 per year

Write a Python program to determine if the applicant is qualified for the loan.

3. Calculate a household's electricity bill based on units consumed using the following rates:

- | | |
|--------------------------------------|--------------------------------------|
| • Up to 100 units → \$0.10 per unit | • 301 to 500 units → \$0.20 per unit |
| • 101 to 300 units → \$0.15 per unit | • Above 500 units → \$0.25 per unit |

Sheet 2: Solutions

1-Program to check if a given year is a Leap Year

Ask the user to enter their birth year

```
year = int(input("Enter your birth year: "))
```

Check leap year condition

```
if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
```

```
    print("The given year is a Leap Year")
```

```
else:
```

```
    print("The given year is not a Leap Year")
```

```
Enter your birth year: 2000  
The given year is a Leap Year
```

```
Enter your birth year: 2010  
The given year is not a Leap Year
```

Sheet 2: Solutions

2- Program to check loan eligibility

Get user input

```
age = int(input("Enter your age: "))
```

```
income = float(input("Enter your annual income: "))
```

Check eligibility

```
if age >= 21 and income >= 25000:
```

```
    print("The applicant is qualified for the loan.")
```

```
else:
```

```
    print("The applicant is not qualified for the loan.")
```

```
Enter your age: 25
```

```
Enter your annual income: 4000
```

```
The applicant is not qualified for the loan.
```

3-Program to calculate electricity bill based on units consumed

```
units = float(input("Enter the number of units consumed: "))
bill = 0

if units <= 100:
    bill = units * 0.10
elif units <= 300:
    bill = units * 0.15
elif units <= 500:
    bill = units * 0.20
else:
    bill = units * 0.25

print(f"The total electricity bill is: ${bill}")
```

```
Enter the number of units consumed: 1000
The total electricity bill is: $205.0
```